



*Department of Computer Science and Engineering (CSE)  
Semester: (Fall, Year: 2025), B.Sc. in CSE*

---

# **Distributed Contest Management System**

---

*Course Title: Integrated Design Project 1 (IDP 1)  
Course Code: CSE 324  
Section: 223 D5*

## Students Details

| <b>Name</b>       | <b>ID</b> |
|-------------------|-----------|
| Nirjhar Das       | 212002076 |
| Md. Shahriar Alam | 213002036 |
| Kaniz Fatema      | 223002094 |

*Submission Date: November 3, 2025  
Course Teacher's Name: Umme Habiba*

[For teachers use only: **Don't write anything inside this box**]

| <u><b>Lab Project Status</b></u> |                   |
|----------------------------------|-------------------|
| <b>Marks:</b>                    | <b>Signature:</b> |
| <b>Comments:</b>                 | <b>Date:</b>      |

# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                              | <b>2</b>  |
| 1.1      | Overview . . . . .                               | 2         |
| 1.2      | Problem Domain . . . . .                         | 2         |
| 1.3      | Motivation . . . . .                             | 3         |
| 1.4      | Objective . . . . .                              | 3         |
| <b>2</b> | <b>Literature Review</b>                         | <b>5</b>  |
| <b>3</b> | <b>Methodology</b>                               | <b>6</b>  |
| 3.1      | SRS . . . . .                                    | 6         |
| 3.1.1    | System Requirements . . . . .                    | 6         |
| 3.1.2    | Functional Requirements . . . . .                | 7         |
| 3.1.3    | Non-Functional Requirements . . . . .            | 7         |
| 3.2      | SDLC . . . . .                                   | 8         |
| 3.3      | DFD . . . . .                                    | 9         |
| 3.3.1    | Level 0 DFD . . . . .                            | 9         |
| 3.3.2    | Level 1 DFD . . . . .                            | 9         |
| 3.3.3    | Data Stores and Data Flow . . . . .              | 10        |
| 3.4      | UML Use Case Diagram . . . . .                   | 11        |
| 3.5      | UML Sequence and Communication Diagram . . . . . | 12        |
| 3.6      | UML Class Diagram . . . . .                      | 12        |
| <b>4</b> | <b>Conclusion</b>                                | <b>14</b> |
| 4.1      | Discussion . . . . .                             | 14        |
| 4.2      | Limitation . . . . .                             | 14        |
| 4.3      | Future Work . . . . .                            | 15        |
|          | <b>References</b>                                | <b>16</b> |

# Chapter 1

## Introduction

### 1.1 Overview

The **Distributed Contest Management System (DCMS)** is a system designed to run programming contests within a school or university LAN environment. In most existing contests, all the code that students write is sent to a single computer (the server) to be compiled and evaluated, which can slow down the server and cause long waiting times for participants.

DCMS works differently: each participant's computer compiles and executes their own code, and only the results are sent to the central system. This approach offers several benefits:

- Students receive feedback faster, as their submissions are processed locally.
- The contest remains responsive even when many students participate simultaneously.
- Organizers can manage the contest more easily, with fewer manual steps and reduced risk of errors.

In short, DCMS aims to make programming contests faster and easier to manage for everyone involved.

### 1.2 Problem Domain

Running programming contests in schools or universities can be challenging. Typically, contests are managed using a single centralized server or a combination of separate tools such as spreadsheets, scripts, and manual record-keeping. [1, 2] This can lead to several issues for all stakeholders.

**Technical Problems:** The main contest server can become overloaded when many students submit code at the same time. [2] This increases response time, reduces overall system performance, and in extreme cases may cause system failures. [2]

**Organizational Problems:** Teachers or organizers often need to update scores, verify submissions, and manage contest data manually, which is time-consuming and error-prone. [1]

**Participant Problems:** Students may have to wait a long time to see whether their code passes the test cases. [1] Inconsistent response times or system behavior can feel unfair and can negatively affect the contest experience. [2]

Overall, current contest management practices can be slow, confusing, and difficult to scale, highlighting the need for a more efficient and reliable solution.

## 1.3 Motivation

DCMS is a valuable project because it involves designing and implementing a real-world distributed system to address practical challenges in programming contest management. Traditional systems rely heavily on a single server, which can become a bottleneck or single point of failure under heavy load. [2] By distributing the workload across participant machines, DCMS has the potential to improve performance and ease of organisation of a contest.

The key motivations for this project are:

- Gaining hands-on experience with distributed computing, network communication, and client–server architecture.
- Implementing fault-tolerant and scalable mechanisms that can handle multiple participants efficiently.
- Creating a secure and reliable environment for executing untrusted code safely on participant machines.
- Developing an intuitive interface and automated tools that simplify contest management for organizers.
- Learning how to monitor and log system performance across multiple nodes in real time.

## 1.4 Objective

The main objectives of the DCMS project are:

- To design a robust, fault-tolerant client–server architecture that efficiently manages communication between a central server and multiple distributed worker nodes.
- To reduce server-side queue times and compilation bottlenecks by distributing compilation and execution tasks to participant machines.

- To provide an intuitive interface for participants to submit code, view problems, and check results.
- To safeguard the integrity of the contest by preventing participants from accessing unauthorized resources, interfering with other submissions, or gaining an unfair advantage.
- To generate detailed evaluation reports for each submission, including compilation errors, runtime errors, test case results, and execution time/memory usage.
- To design a comprehensive logging and monitoring system for tracking submission history, system performance, and debugging information across all distributed components.

# Chapter 2

## Literature Review

Over the years, several contest management systems have been developed to facilitate programming competitions, such as **DOMjudge**, **Codeforces Polygon**, and **PC<sup>2</sup>** (Programming Contest Control System). While these tools are effective in online or small-scale local contexts, they rely heavily on centralized architectures. This leads to common issues like server overload, latency, and limited scalability.

### Existing Systems and Their Limitations

- **DOMjudge:** A widely used open-source contest management system that allows automated judging [3]. However, it requires a powerful central server and multiple judge hosts, which increases setup complexity.
- **PC<sup>2</sup>:** Developed by California State University, PC<sup>2</sup> has been used in ICPC regional contests [4]. Despite its robustness, it still depends on centralized judgment and manual configuration for distributed setups.
- **Codeforces and HackerRank:** Online platforms offering large-scale contests, but they are cloud-dependent and unsuitable for LAN-based or offline environments [5].

### Research Trends

Recent studies on distributed computing highlight the efficiency of decentralizing computational workloads to improve fault tolerance and responsiveness [6]. In the context of educational contests, applying distributed principles can create more scalable and resilient systems. This project leverages such principles, implementing peer-assisted judgment and local compilation to improve contest throughput and interactivity.

In summary, while previous works focus on centralized architectures with high hardware dependency, **DCMS** introduces a hybrid distributed model that emphasizes scalability, reduced latency, and enhanced autonomy for each participant node.

# Chapter 3

## Methodology

### 3.1 SRS

A clear view of the system's requirements is crucial for developing a reliable and efficient Distributed Contest Management System (DCMS). The requirements are classified into system, functional, and non-functional aspects to define both what the system must do and how well it should perform.

#### 3.1.1 System Requirements

DCMS is going to operate within a university LAN environment, utilizing a distributed architecture to optimise performance & scalability. In order to achieve that the key system-level requirements identified are as follows:

- **Decentralised Control Structure:** The system must support distributed execution and evaluation of user submissions across participant nodes. Each participant's machine acts as a lightweight worker node capable of compiling, executing, and judging code locally, thereby minimising dependence on the central server.
- **Concurrency and Synchronisation:** The system must handle multiple simultaneous user submissions and real-time updates without conflicts. Participant interfaces should automatically synchronise with the admin system.
- **Network-Based Coordination:** The system should maintain consistent communication between the central controller (admin) and all participant nodes through the LAN. Reliable data exchange protocols are required to ensure low-latency message delivery and seamless coordination during contests.
- **Cross-Platform Compatibility:** The system must be operable across major operating systems (Windows, Linux, and macOS) with minimal configuration. The design should be independent of platform-specific dependencies.
- **Fault Recovery:** The system should detect node failures or disconnections and continue operation without global disruption. Any missing results or updates must be logged and resynchronised automatically once the node reconnects.

### 3.1.2 Functional Requirements

The functional requirements are operations and interactions between system components. DCMS should fulfil the following functionalities:

- **Automated Problem Statement Update:** When the admin uploads or modifies a problem, it must automatically appear on all participant interfaces without manual refresh or restart.
- **Automated Constraint Configuration:** The system must dynamically propagate problem-specific constraints, such as time limits, memory limits, or input/output formats, to participant systems during the runtime of the contest.
- **Code Submission and Local Testing:** Participants must be able to write, compile, and test their solutions directly through the local interface. Compilation and test execution occur locally, and only results are sent to the server.
- **Automated Evaluation on Participant End:** Each participant node must evaluate submitted programs against predefined hidden test cases locally, generating verdicts such as “Accepted”, “Wrong Answer”, “Time Limit Exceeded”, or “Runtime Error”.
- **Result Synchronisation:** After evaluation, the results must be securely transmitted to the central server, which aggregates and updates the scoreboard in real-time.
- **Submission Tracking and Logging:** Every submission must be timestamped, versioned, and logged for both participant and admin reference. The admin dashboard should allow viewing submission history and verdicts per user.
- **Admin Control and Monitoring:** The admin should be able to broadcast commands (e.g., start contest, end contest, disable submission) to all participants simultaneously and monitor system health across the network.

### 3.1.3 Non-Functional Requirements

The non-functional requirements specify the quality attributes that ensure DCMS is efficient, maintainable, and user-friendly.

- **Code Editor for Participant:** A lightweight, integrated code editor must be available for participants, supporting syntax highlighting, basic auto-indentation, and compile/run shortcuts to minimize context switching.
- **Performance and Scalability:** The system must efficiently handle dozens of simultaneous nodes without noticeable degradation in speed or responsiveness. Latency between problem updates and client synchronization should remain under one second in optimal LAN conditions.

- **Fault Tolerance and Reliability:** In case of hardware or network failure, the system must recover gracefully. Logs and submission data should be preserved to prevent data loss. Retransmission mechanisms should ensure that no submission is lost due to transient network issues.
- **Usability and Accessibility:** The interfaces for both participants and administrators must be intuitive and consistent. The system should minimize cognitive load by providing clear feedback messages, status indicators, and contextual help.
- **Security and Integrity:** The system must enforce isolation of participant environments to prevent cheating or tampering with test cases. File I/O and network access from participant programmes should be sandboxed or restricted as appropriate.
- **Maintainability and Extensibility:** The architecture should be modular, allowing future integration of additional judging features, new programming languages, or external judging modules without major redesign.
- **Portability:** The system's deployment should require minimal setup, with configuration scripts or containerized environments (e.g., Docker) to simplify installation across different OS and hardware setups.

## 3.2 SDLC

SDLC is a process followed for a software project that consists of a detailed plan describing how to develop, maintain, replace, alter, or enhance specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

Here, we select some criteria that are important and suitable for *DCMS*. Then, we assign priorities to each criterion based on its importance. Finally, we chose the most suitable model *SDLC* based on the priority we set for each criterion.

Table 3.1: Comparison matrix with different models

| Priority   | Criteria                   | Waterfall | V-shape | Iterative | Spiral | Agile | Prototype |
|------------|----------------------------|-----------|---------|-----------|--------|-------|-----------|
| 5          | Well known requirement     | Yes       | Yes     | No        | No     | No    | No        |
| 4          | Technological knowledge    | Yes       | Yes     | No        | No     | No    | No        |
| 6          | Efficiency                 | No        | Yes     | Yes       | Yes    | No    | Yes       |
| 2          | Risk analysis              | No        | No      | No        | Yes    | No    | No        |
| 5          | User testing ability       | No        | No      | Yes       | Yes    | Yes   | Yes       |
| 2          | Dependability and Security | No        | No      | Yes       | Yes    | No    | No        |
| 2          | Time consuming             | Yes       | Yes     | No        | Yes    | No    | No        |
| Total = 26 | Over all                   | 11        | 17      | 13        | 17     | 5     | 11        |

In Fig 3.1 we see, *Waterfall* scores 11, *V – Shaped* scores 17, *Iterative* scores 13, *Spiral* scores 17, *Agile* scores 5, and *Prototype* scores 11. The most important criteria for our project — well-known requirements, efficiency, and user testing ability — are best supported by the *V – Shaped* and *Spiral* models.

We prefer the **\*\*V-shaped model\*\*** as the most suitable *SDLC* model for the *DCMS* project over *Spiral*. Although both models have scored 17, *Spiral* focuses too much on

risk analysis, which is unnecessary for *DCMS*. The model *V – Shaped* provides a structured approach with clear verification and validation phases, which aligns well with our priorities and ensures the development of a reliable and efficient contest management system.

### 3.3 DFD

Data Flow Diagrams (DFD) are used to visually represent how data moves through the Distributed Contest Management System (DCMS). They help in understanding the interaction between external entities, internal processes, and data stores at different levels of abstraction.

#### 3.3.1 Level 0 DFD

The Level 0 Data Flow Diagram illustrates the DCMS as a single high-level process interacting with external entities. This diagram defines the system boundary and shows how data enters and exits the system without revealing internal details.

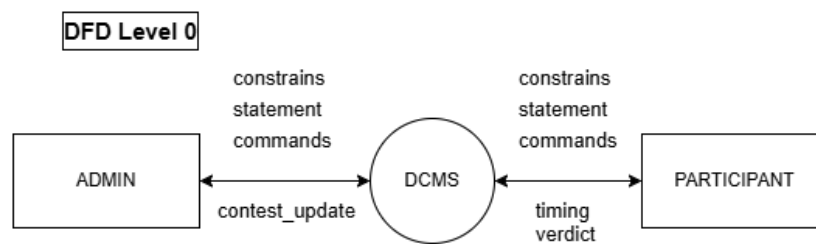


Figure 3.1: Level 0 Data Flow Diagram of DCMS

In this diagram, the main external entities are the **Admin** and the **Participants**. The Admin provides problem statements, contest configurations, and control commands, while Participants submit source code and receive evaluation results. The DCMS core process manages all interactions, ensuring smooth coordination between participants and the contest controller.

#### 3.3.2 Level 1 DFD

The Level 1 Data Flow Diagram provides a detailed view of the internal processes of the Distributed Contest Management System (DCMS) by decomposing the core system into multiple functional components.

As shown in Fig. 3.2, the DCMS interacts with two primary external entities: the **Admin** and the **Participant**. The Admin is responsible for managing contest-related data, including problem definitions and constraints, while Participants submit solutions and receive feedback.

Within the system, the **Problem Distributor** process receives problem statements from the Admin and distributes them to all participants. The **Constraint Setter** con-

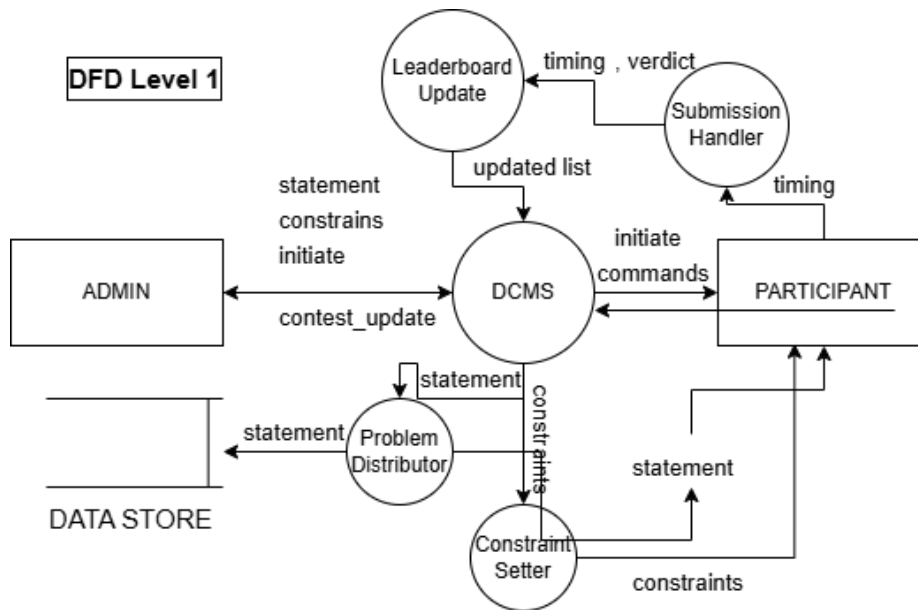


Figure 3.2: Level 1 Data Flow Diagram of DCMS

figures execution constraints such as time limits and memory limits and ensures these constraints are applied consistently during evaluation.

The **Submission Handler** manages source code submissions from participants and forwards them for evaluation. Once submissions are processed, the **Leaderboard Update** process aggregates the results and updates the contest standings accordingly.

All these processes are coordinated through the central **DCMS** core, ensuring synchronized communication, accurate result processing, and real-time contest updates. This structured decomposition highlights how DCMS efficiently manages contest operations while distributing responsibilities across specialized sub-processes.

### 3.3.3 Data Stores and Data Flow

In the Level 1 DFD, data storage and movement are logically managed through coordinated system processes rather than explicit standalone databases. Problem-related data provided by the **Admin** is handled by the **Problem Distributor**, while execution constraints are processed by the **Constraint Setter**. These processes ensure that all participants receive consistent problem statements and contest rules.

Submission data originating from the **Participant** is processed by the **Submission Handler**, which manages submission flow and evaluation status. Once verdicts are generated, the **Leaderboard Update** process aggregates the results and reflects them in the contest standings.

Data flows between the Admin, Participant, and internal DCMS processes occur in a synchronized manner through the central **DCMS** core. This structured flow ensures timely problem distribution, controlled submission handling, and accurate result updates, while maintaining system responsiveness and reducing central processing overhead.

### 3.4 UML Use Case Diagram

The UML Use Case Diagram represents the functional behaviour of the Distributed Contest Management System (DCMS) by illustrating the interactions between system users and the system functionalities. It provides a high-level overview of the services offered by DCMS without exposing internal implementation details.

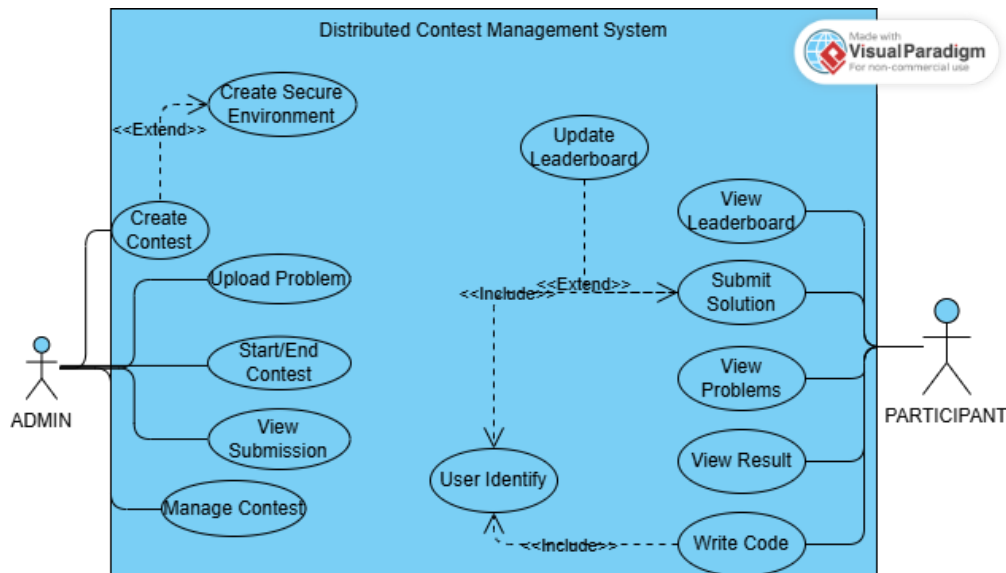


Figure 3.3: UML Use Case Diagram of DCMS

As shown in Fig. 3.3, the system involves two primary actors: the **Admin** and the **Participant**. The Admin is responsible for managing the overall contest environment. This includes creating and managing contests, uploading problem statements, starting or ending contests, and viewing participant submissions. The **Create Secure Environment** use case extends the **Create Contest** functionality to ensure a controlled and secure execution environment during contest setup.

The Participant interacts with the system by viewing available problems, writing source code, submitting solutions, and viewing evaluation results and leaderboard standings. The **User Identify** use case is included in core participant actions to ensure proper authentication and authorization before accessing system features.

The **Submit Solution** use case includes result processing and extends to the **Update Leaderboard** functionality, ensuring that successful submissions are reflected in the contest rankings. Similarly, writing code and viewing results are linked through included relationships to maintain consistent user identity verification.

Overall, the use case diagram clearly defines the responsibilities of each actor and demonstrates how DCMS coordinates contest management, secure participation, and result evaluation in a structured and user-centric manner.

### 3.5 UML Sequence and Communication Diagram

The UML sequence diagram illustrates the time-ordered interaction between system components during the submission and evaluation of a participant's solution in the Distributed Contest Management System (DCMS).

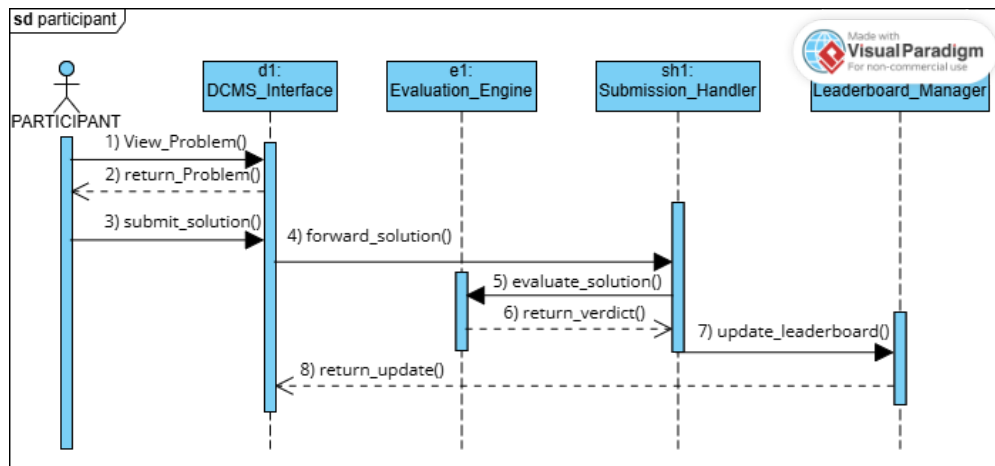


Figure 3.4: UML Sequence Diagram for Submission and Evaluation Process

As shown in Fig. 3.4, the interaction begins when the participant submits a solution through the **DCMS Interface**. The submission is forwarded to the **Submission Handler**, which is responsible for managing and validating incoming submissions.

The validated submission is then passed to the **Evaluation Engine**, where the source code is compiled and executed against predefined constraints and test cases. After evaluation, the resulting verdict is sent to the **Leaderboard Manager**, which updates the contest rankings accordingly.

Finally, the evaluation result is propagated back through the DCMS Interface and presented to the participant. This sequence ensures an orderly, efficient, and synchronized workflow for code submission, evaluation, and result dissemination within the system.

### 3.6 UML Class Diagram

The UML class diagram represents the static structure of the Distributed Contest Management System (DCMS). It shows the main system classes, their attributes, methods, and the relationships between them. This diagram helps in understanding how data and responsibilities are organized within the system.

As illustrated in Fig. 3.5, the DCMS consists of several core classes. The base classes are specialized into **Admin** and **Participant**, representing different system roles. The Admin class is responsible for contest creation, problem management, and system control, while the Participant class handles code submission and result viewing.

The **Contest** class manages contest-related information such as contest status and timing, which is instantiated by the **Admin** class. Participant submissions are handled

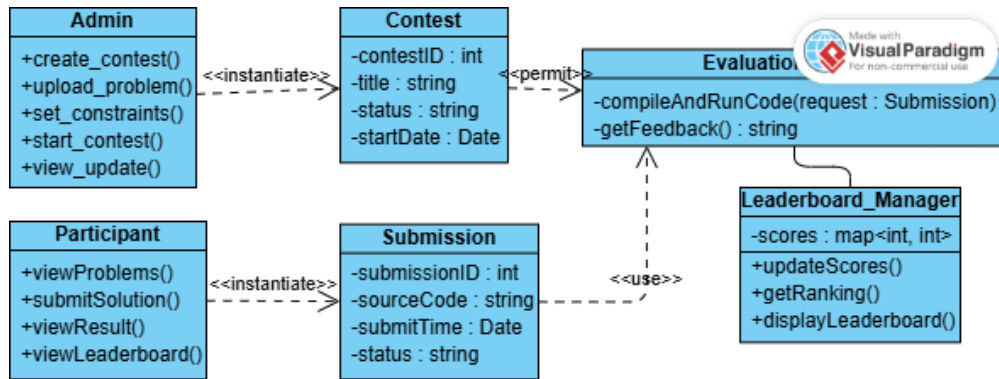


Figure 3.5: UML Class Diagram of DCMS

through the **Submission** class, which contains details about the source code, submission time, and evaluation status and depends on the **Contest** class, which permits the **EvaluationEngine** class to work.

The **EvaluationEngine** class is responsible for compiling and executing submitted code based on predefined constraints, while the **Leaderboard** class maintains ranking information and updates participant scores accordingly. These classes interact to ensure smooth contest execution, accurate evaluation, and real-time result updates.

Overall, the UML class diagram provides a clear structural overview of DCMS and demonstrates how different components collaborate to deliver a distributed and efficient contest management system.

# Chapter 4

## Conclusion

### 4.1 Discussion

This project involved us with the design and analysis of a DCMS aimed at improving the efficiency of programming contests conducted within a university or institutional LAN environment. Unlike traditional judging systems, DCMS distributes the compilation and execution workload to participant machines, significantly reducing server-side bottlenecks and latency.

Through the proposed architecture and SRS, the project shows how simple distributed computing principles can be effectively applied to an educational contest setting, so the system supports real-time synchronization, automated evaluation, and centralized monitoring while maintaining participant autonomy. By clearly defining functional and non-functional requirements, DCMS ensures reliability, usability, and security throughout the contest lifecycle.

The selection of the V-shaped SDLC model further strengthened the development approach by aligning verification and validation activities with each development phase. [7] This ensured that system requirements were consistently met and tested, contributing to a more structured and dependable system design. [7]

Overall, DCMS addresses the key limitations of existing contest management systems and provides a practical alternative suitable for offline or LAN-based programming competitions.

### 4.2 Limitation

Despite the advantages, the proposed theory has several limitations. First, since code compilation and execution occur on participant machines, the system depends on the runtime environments on each node. Hence, the differences in hardware performance may lead to slight variations in execution time measurement.

Secondly, ensuring strong security and sandboxing on all participant machines can be challenging, especially in environments without strict administrative control. While the system is designed to restrict unauthorised access and unfair practices, implement-

ing robust isolation mechanisms may require additional effort.

Finally, the current design focuses primarily on LAN-based environments. As a result, it may not perform optimally over unstable or wide-area networks without further enhancements to fault handling and synchronization mechanisms.

## **4.3 Future Work**

There are several directions in which the DCMS project can be extended in the future.

Support for additional programming languages and more advanced judging features – such as plagiarism detection, interactive problems, and partial scoring – can further enhance the system. Another potential improvement is extending the system to support hybrid or cloud-assisted architectures, allowing DCMS to operate efficiently beyond LAN environments.

Additionally, developing a more advanced analytics dashboard for administrators, including performance metrics and visual reports, would improve monitoring and decision-making during contests. With these improvements, DCMS can evolve into a more comprehensive and production-ready contest management platform.

# References

- [1] Miguel A. Revilla, Salvador Manzoor, and Rujia Liu. The UVa online judge experience. *Olympiads in Informatics*, 2:191–198, 2008.
- [2] Hiroyuki Ito, Masakazu Soshi, Naoya Sato, Masaki Nishi, Yoshiki Higo, and Shinji Kusumoto. Online judge system: Requirements, architecture, and experiences. *International Journal of Software Engineering and Knowledge Engineering*, 32(05):661–690, 2022.
- [3] DOMjudge Team. Domjudge team manual, release 8.0.0dev. <https://docs.icpc.global/wp-content/uploads/2021/09/DOMJudge.Team-Manual.WF2021.pdf>, 2021. Accessed: 2025-11-03.
- [4] California State University, Sacramento. Pc<sup>2</sup> version 8.5 administrator’s guide. <https://acm2007.cct.lsu.edu/pc2/pc2v85AdminGuide.htm>, 2003. Accessed: 2025-11-03.
- [5] Kevin K. F. Yuen, Dennis Y. W. Liu, and Hong Va Leong. Competitive programming in computational thinking and problem-solving education. *International Journal of Educational Technology in Higher Education*, 18(1):1–21, 2021.
- [6] Ahmed Bentiba, Mohamed Al Mansoori, and Jamal J. Zemerly. Wacs: an alternative to pc<sup>2</sup> for programming contests. In *Proceedings of Saudi International Conference on Information Technology (SIIT '09)*, pages 511–518. KAUST / IEEE, 2009.
- [7] GeeksforGeeks. SDLC V-Model - software engineering. <https://www.geeksforgeeks.org/software-engineering/software-engineering-sdlc-v-model/>, 2018. Accessed: 2025-12-24.